



# Sweet Crumbs

Baking Blog — Full Setup Guide

|          |        |             |           |
|----------|--------|-------------|-----------|
| Supabase | Vercel | Google Auth | OmniRoute |
|----------|--------|-------------|-----------|

A complete walkthrough of building a production baking blog  
with database, authentication, deployment, and browser automation.

---

## Table of Contents

|                                                     |    |
|-----------------------------------------------------|----|
| Table of Contents .....                             | 2  |
| 1. Supabase Project Setup .....                     | 3  |
| Project Configuration .....                         | 3  |
| Table Schema .....                                  | 3  |
| 2. Frontend Integration .....                       | 4  |
| Data Flow .....                                     | 4  |
| REST API Call Pattern .....                         | 4  |
| Key Learning .....                                  | 4  |
| 3. Vercel Deployment .....                          | 5  |
| 4. Google OAuth Setup .....                         | 6  |
| Authentication Flow .....                           | 6  |
| Setup Steps .....                                   | 6  |
| 5. Bugs Fixed .....                                 | 7  |
| Bug 1: Broken Images (404) .....                    | 7  |
| Bug 2: Auth Redirect to localhost .....             | 7  |
| Bug 3: Duplicate const Declaration (CRITICAL) ..... | 7  |
| Bug 4: User Profile Not Showing .....               | 7  |
| 6. Playwright MCP Setup .....                       | 8  |
| Comparison .....                                    | 8  |
| 7. OmniRoute (AI Gateway) .....                     | 9  |
| Fixing 401 Token Errors .....                       | 9  |
| Key Takeaways .....                                 | 10 |

## 1. Supabase Project Setup

Supabase provides a PostgreSQL database with a built-in REST API (PostgREST). We created a project, defined a recipes table, configured security, and seeded 4 recipes.

### Project Configuration

| Property     | Value                                    |
|--------------|------------------------------------------|
| Project Name | baking-blog-recipes                      |
| Project ID   | bynfesgbvgcmkpnwysil                     |
| Region       | ap-south-1 (Mumbai)                      |
| URL          | https://bynfesgbvgcmkpnwysil.supabase.co |
| Tier         | Free                                     |

### Table Schema

```
CREATE TABLE recipes (  
  id          TEXT PRIMARY KEY,  
  title       TEXT NOT NULL,  
  image       TEXT,  
  category    TEXT NOT NULL,  
  "prepTime"  INTEGER NOT NULL,  
  "cookTime"  INTEGER NOT NULL,  
  servings    INTEGER NOT NULL,  
  difficulty   TEXT NOT NULL,  
  description  TEXT,  
  ingredients JSONB DEFAULT '[]'::jsonb,  
  steps       JSONB DEFAULT '[]'::jsonb,  
  created_at  TIMESTAMPTZ DEFAULT now()  
);
```

RLS enabled with public read policy. Data API grants configured for **anon** role.

## 2. Frontend Integration

The frontend fetches recipes from Supabase using plain `fetch()` — no SDK needed. Each page loads data asynchronously with loading states and error handling.

### Data Flow



### REST API Call Pattern

```
const response = await fetch(
  `${SUPABASE_URL}/rest/v1/recipes?select=*`,
  {
    headers: {
      'apikey': SUPABASE_ANON_KEY,
      'Authorization': `Bearer ${SUPABASE_ANON_KEY}`
    }
  }
);
```

### Key Learning

- Endpoint: `{SUPABASE_URL}/rest/v1/{table}?select=*`
- Headers required: `apikey` and `Authorization` (both use the anon key)
- RLS policies control which rows are visible
- Data API grants control whether the table is accessible at all

### 3. Vercel Deployment

The baking blog is deployed on Vercel as a static site. Deployments happen instantly via the Vercel CLI.

| Property       | Value                                |
|----------------|--------------------------------------|
| Production URL | https://baking-blog-three.vercel.app |
| Project ID     | prj_Tn5Grdcjfm8csMr55dpl9z6xqmnL     |
| Deploy Command | npx vercel --prod --yes              |

## 4. Google OAuth Setup

Google authentication was set up across three systems: Google Cloud Console, Supabase Auth, and the frontend JavaScript.

### Authentication Flow



### Setup Steps

1. Google Cloud: OAuth consent screen (External) → Web Client ID → Redirect URI
2. Supabase: Auth → Providers → Google → Paste Client ID + Secret → Save
3. Remove localhost from redirect URLs. Add production URL only.

## 5. Bugs Fixed

### Bug 1: Broken Images (404)

**Problem:** Vanilla cupcakes and banana bread images returned 404 from Unsplash.

**Fix:** Replaced with working Unsplash photo IDs.

### Bug 2: Auth Redirect to localhost

**Problem:** After Google login, browser redirected to localhost:3000 instead of production.

**Fix:** Hardcoded production URL in auth redirect + removed localhost from Supabase URLs.

### Bug 3: Duplicate const Declaration (CRITICAL)

**Problem:** Both auth.js and script.js declared const SUPABASE\_URL — crashed entire script, no recipes loaded.

**Fix:** Removed duplicate from script.js, kept only in auth.js.

**Lesson:** Multiple <script> tags share scope. Duplicate const = TypeError.

### Bug 4: User Profile Not Showing

**Problem:** Supabase API returns raw\_user\_meta\_data, not user\_metadata.

**Fix:** Check both field names + JWT decode fallback.

## 6. Playwright MCP Setup

Playwright MCP enables browser automation for testing. The key insight: shell script calls create new HTTP sessions (losing browser context), while a registered MCP server uses persistent stdio connections.

### Comparison

| Approach      | Transport           | Context            | Result               |
|---------------|---------------------|--------------------|----------------------|
| Shell scripts | HTTP (new per call) | Lost between calls | ✗ Blank screenshots  |
| MCP server    | Stdio (persistent)  | Maintained         | ✓ Working automation |



## 7. OmniRoute (AI Gateway)

OmniRoute is a free AI gateway that proxies requests to 36+ providers. It handles OAuth token management for Claude Code with automatic background refresh.

### Fixing 401 Token Errors

When Claude Code shows "401: You need to sign in to use this model":

1. Open OmniRoute dashboard: <http://localhost:20128>
2. Go to Providers
3. Find the Claude/Anthropic provider
4. Click Reconnect — or delete and re-add it
5. Restart Claude Code terminal

*From OmniRoute docs: "OAuth token expired. Auto-refreshed; if stuck, delete + re-auth in Providers."*

## Key Takeaways

1. **Supabase REST API:** Just HTTP GET with an API key header — no SDK needed
2. **RLS Policies:** Critical — without them, your data is publicly exposed
3. **Data API Grants:** Separate from RLS — both must be configured
4. **OAuth Redirect URIs:** Must exactly match — even trailing slashes matter
5. **const Scope:** Multiple script tags share scope — duplicate const = TypeError
6. **User Metadata:** Supabase API returns raw\_user\_meta\_data, not user\_metadata
7. **JWT Fallback:** Tokens contain user data — useful when API calls fail
8. **MCP Transport:** Stdio maintains persistent connections; HTTP does not
9. **OmniRoute Tokens:** Auto-refresh but can get stuck — reconnect in dashboard



### Sweet Crumbs Baking Blog — Session Summary

Generated from a Claude Code + OmniRoute session